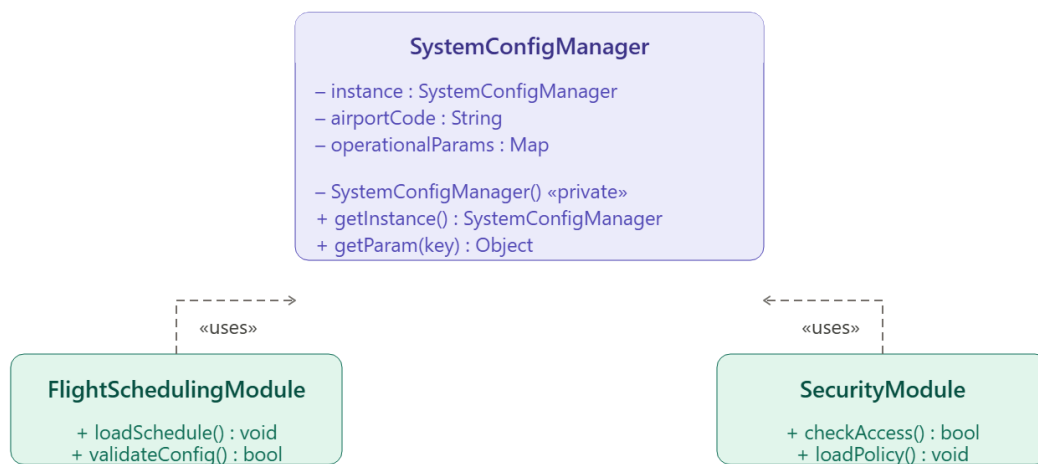


## Pattern 1: Singleton — System Configuration Manager

### Justification

The Airport Management System must maintain a single, authoritative configuration state — covering runway codes, gate numbering schemes, operational parameters, and role definitions (BR\_02). If multiple instances of the configuration object existed, concurrent modifications by different modules could produce conflicting settings and unpredictable behaviour. The Singleton pattern guarantees that exactly one `SystemConfigManager` instance is created and shared across all subsystems (flight scheduling, security, HR, ground handling), ensuring configuration changes made by an Administrator are immediately and consistently visible everywhere without synchronisation overhead

### Class Diagram

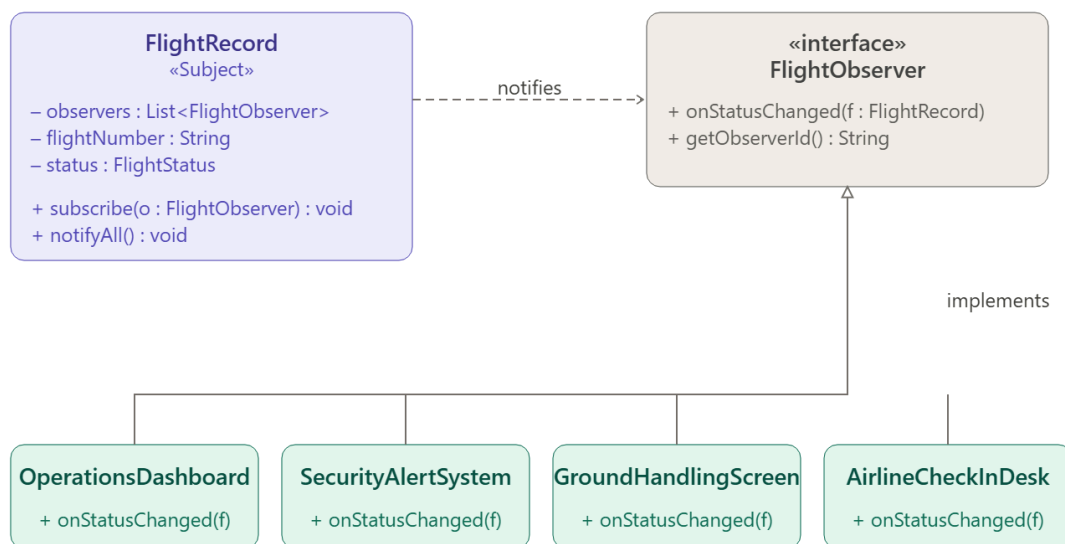


## Pattern 2: Observer — Flight Status Notification

### Justification

Flight status changes (delays, gate modifications, cancellations) must be broadcast in real time to multiple independent stakeholders: Operations Staff dashboards (BR\_08), Security alert systems (BR\_15), Ground Handling arrival screens (BR\_22), and Airline check-in desks (BR\_28). Hard-coding direct method calls from FlightRecord to every consumer would create tight coupling and make adding new subscribers require modifying core domain classes. The Observer pattern decouples the subject (FlightRecord) from its observers, so any module implementing FlightObserver is automatically notified on state change — satisfying the non-functional requirement for real-time updates across all roles.

### Class Diagram

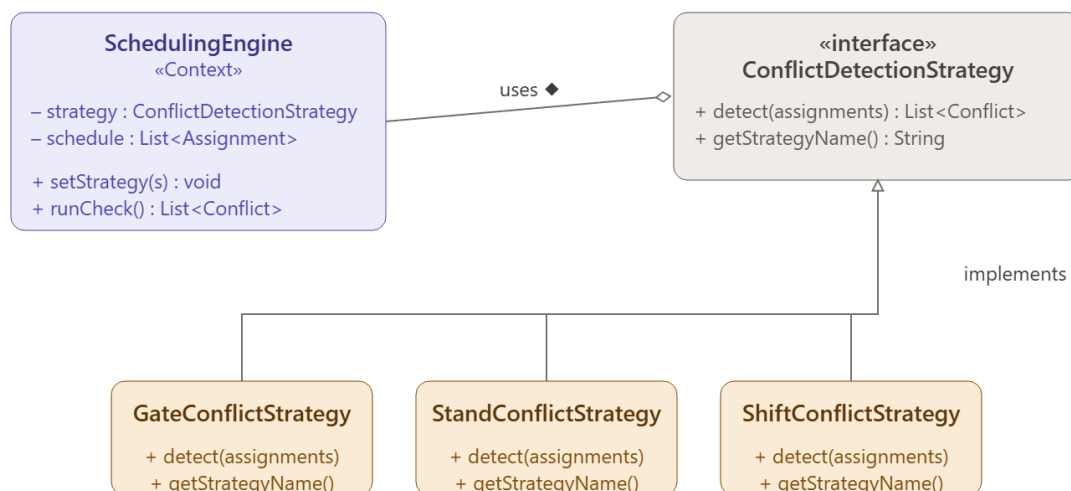


## Pattern 3: Strategy — Conflict Detection Algorithm

### Justification

The system must detect scheduling conflicts between flights, gates, and stands (BR\_10) and similar overlaps in staff shift scheduling (BR\_37). Different operational contexts demand different detection algorithms: gate conflicts need time-window overlap checks, stand conflicts must also factor in aircraft type, and HR shift conflicts involve employee availability calendars. If the detection logic were embedded directly in the scheduling modules, switching or adding a new algorithm would require modifying stable, tested code. The Strategy pattern encapsulates each algorithm behind a common ConflictDetectionStrategy interface, letting the SchedulingEngine swap strategies at runtime without changing the engine itself.

### Class Diagram



## Pattern 4: Facade — Ground Operations Facade

### Justification

Ground Handling staff must coordinate several interdependent subsystems in a single operational cycle: viewing the arrival schedule (BR\_22), tracking baggage operations (BR\_23), assigning apron services such as refuelling and catering (BR\_24), and confirming all ground services before departure (BR\_27). Exposing each subsystem directly to the UI layer would force the Ground Handling module to understand and orchestrate all of them independently. The Facade pattern places a single GroundOperationsFacade in front of the four subsystem services, providing a clean, high-level API that hides subsystem complexity and reduces the risk of incomplete or inconsistent ground service cycles.

### Class Diagram

